RTSP Link Authentication: Credentials & APIs

How the script-server gets a playable RTSP URL from the Camview (Dahua DSS) DVR servers · prepared 10 Oct 2026 from the code in `script-server/`

Confidential: this file contains the live DVR password. Do not commit it, publish it or share it outside the team.

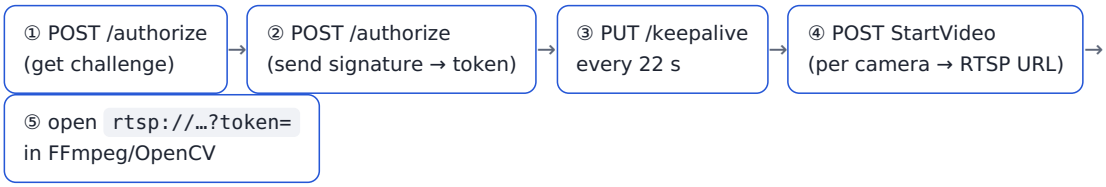
1. Credentials

Item	Value	Where it is set
Username	`system`	`CAMVIEW_AUTH_USER` in `script-server/.env`
Password (live)	`Coreiseven@@`	`CAMVIEW_AUTH_PASS` in `script-server/.env` (changed 29 Sep 2026)
Old password (not valid)	`Sasd@12345`	still hard-coded as the fallback default in many scripts; used only if `.env` is not loaded
Per-DVR password override	none set right now	`CAMVIEW_AUTH_PASS_d15=<pwd>` applies to `d15.camview.io` (read by the token server, which does all logins)
Client type	`WINPC_V1`	fixed in code
DVR servers	`d1, d2, d7, d8, d11, d14, d15, d16, d23, d24, d25` .camview.io	taken per camera from `camera_links_rtsp.cctv_url`, which holds the DVR host name (e.g. `d14.camview.io`; some rows are upper-case like `D25.camview.io`)

The same `system` account is used on every DVR. **Camview allows only one active session per username**: a new login kills the previous token. That is why all logins go through one token server (section 4).

Checked on 10 Oct 2026: there is no other DVR/RTSP user id or password. No per-DVR override is set, and no row in `camera_links_rtsp` or `camera_links_hls` has a `user:pass@` inside its URL. `DB_USER` / `DB_PASSWORD` in `.env` are for MySQL, not cameras.

2. Flow overview



Steps ①–③ run once per DVR in the token server. Step ④ runs per camera on each worker. Base URL for every call: `https://<server>/admin/API/`. The token server and classification skip TLS certificate checks (`verify=False`); detection's StartVideo call uses normal certificate checks.

3. Camview DVR APIs

① Login, step 1: get challenge

```
POST https://dXX.camview.io/admin/API/accounts/authorize
Content-Type: application/json

{ "userName": "system", "ipAddress": "", "clientType": "WINPC_V1" }
```

Response (HTTP 401 or 200) gives `realm`, `randomKey`, `publickey`, plus a `Set-Cookie` header that must go back in step 2.

② Login, step 2: send signature, get token

Signature is a five-round MD5 chain:

```
t1 = md5(password)
t2 = md5(user + t1)
t3 = md5(t2)
t4 = md5(user + ":" + realm + ":" + t3)
signature = md5(t4 + ":" + randomKey)
```

```
POST https://dXX.camview.io/admin/API/accounts/authorize
Content-Type: application/json
Cookie: <Set-Cookie from step 1>

{ "mac": "", "signature": "<signature>", "userName": "system",
  "randomKey": "<randomKey>", "publicKey": "<publickey>",
  "encryptType": "MD5", "ipAddress": "", "clientType": "WINPC_V1", "userType": "0" }
```

Response: `{"token": "..."}`. This is the **session token**.

Error code	Meaning	What the code does
`1101`	MD5 rejected (needed for d15)	gets a fresh challenge and retries with `"encryptType": "RSA"`
`2002`	account locked (`unlockRemainTime` in response)	stops; token server waits 400 s before trying again (lockout is ~300 s)

③ Keepalive

```
PUT https://dXX.camview.io/admin/API/accounts/keepalive
Content-Type: application/json
X-Subject-Token: <session token>

{}
```

Sent every 22 s. `200` = session alive. `401` = session gone, token dropped, next request logs in again.

④ StartVideo: get the RTSP URL for one camera

```
POST https://dXX.camview.io/admin/API/MTS/Video/StartVideo?token=<session token>
Content-Type: application/json;charset=UTF-8
X-Subject-Token: <session token>

{ "clientType": "WINPC_V1", "project": "PSDK", "method": "MTS.Video.StartVideo",
  "data": {
    "streamType": "1",
    "optional": "/admin/API/MTS/Video/StartVideo?token=<session token>",
    "channelId": "<device_id>$1$0$<camera_no>",
    "dataType": "2",
    "enableRtsps": "0"
  } }
```

- `device_id` and `camera_no` come from the `camera_links_rtsp` row.
- Response `data.url` (and `data.url2`) can hold several URLs joined by `|`. The code splits them; detection also drops URLs whose port is not reachable.
- Response `data.token` is the **stream token** (falls back to the session token if missing).
- `401` here means the session token is dead: the worker reports it to the token server (`/evict`) and fetches a new one. Classification first checks the 401 body, because some firmware returns stream data with a 401.
- Detection caches the result per camera for 180 s (`STARTVIDEO_CACHE_TTL`).

⑤ Final RTSP URL

```
rtsp://<host>:<port>/<path from data.url>?token=<stream token>
```

No username/password goes into the RTSP URL itself; the token is the only auth.

4. Token server (internal API)

`scripts/detection/camview_token_server.py` runs on **worker-0 (192.168.19.30), port 8090** under pm2 as `camview-token-server` **(online, checked today)**. It is the only process that logs in to the DVRs; every worker asks it for the token.

Endpoint	Purpose	Response
`GET /token?server=d14.camview.io`	Return the shared session token; logs in if none is alive. `&force=1` forces a fresh login.	`200 {"token": "..."}` or `503 {"error": "auth failed"}`
`POST /evict` body `{"server": "...", "token": "..."}`	Worker reports a token that got 401. When the body has `token` (detection does this), it is evicted only if it is still the current one, so many reports cause one re-login. Classification sends only `server`, which evicts the current token unconditionally.	`{"ok": true, "evicted": true|false}`
`GET /health`	Status: active sessions, their age, DVRs in backoff	`{"status":"ok","servers":[…],"sessions":{…},"in_backoff":{…}}`

Setting	Default	Meaning
`CAMVIEW_TOKEN_SERVER`	`http://localhost:8090` on worker-0, `http://192.168.19.30:8090` elsewhere	set to `""` to log in directly (single host only)
`CAMVIEW_ALIVE_WINDOW`	88 s	token handed out only if login/keepalive succeeded within this window
`CAMVIEW_AUTH_BACKOFF`	400 s	wait after a failed login before retrying that DVR
`VISION_TIMEOUT_API`	6 s	HTTP timeout for login calls
`TOKEN_SERVER_LOCAL_TTL`	20 s	how long a worker process reuses a token before asking again
`TOKEN_SERVER_FAIL_TTL`	30 s	how long a worker waits after the server says a DVR is unavailable

5. Where it is in the code

File	Role
`scripts/detection/camview_token_server.py`	Login, keepalive, `/token` `/evict` `/health`
`scripts/detection/pipeline/master_pipeline.py`	Detection: `get_server_token()`, `get_rtsp_from_dss()` (StartVideo), `invalidate_server_token()`
`scripts/detection/vision_classification_rtsp.py`	Classification: same token-server client
`scripts/detection/utils/dss_stream.py`, `fetch_rtsp_url.py`	Stand-alone/debug versions that log in directly with the **old** hard-coded password `Sasd@12345`
`script-server/.env`	`CAMVIEW_AUTH_USER`, `CAMVIEW_AUTH_PASS`

6. Things to watch

- Debug scripts (`dss_stream.py` , `fetch_rtsp_url.py` , `test_camera_frame_capture.py` , `test_rtsp_from_excel.py`) do not load `.env` and have the old password hard-coded, so their login now fails (`test_camera_frame_capture.py` uses `CAMVIEW_AUTH_PASS` only if it is exported in the shell). Repeated failed logins can lock the shared `system` account on that DVR for the whole fleet. Update them to read `.env` before using them. With the right password, a direct login would instead kill the shared session and every worker would get 401 until the token server logs in again.
- Repeated bad logins lock the `system` account for ~5 min (code 2002) on that DVR.
- Only the token server (worker-0) logs in, so a password change takes effect after updating `script-server/.env` on worker-0 and running `pm2 restart camview-token-server` . Keep `.env` on the other workers in sync too, for the direct-login fallback.